CIMAT

Homework 4: EDA with MIMIC and Chow-Liu Tree using Normal Variables

Pedro Llerenas [1]

[1] CIMAT pedro.llerenas@cimat.mx

Abstract: We test the MIMIC and Chow-Liu tree EDA's for normal variables on three functions: Sphere, Ackley and Griewank. In particular, the mutual information of two normal random variables is given by $-\frac{1}{2}\log(1-\rho^2)$, where ρ is the Pearson correlation coefficient.

Keywords: Mutual Information; Normal Distribution; Estimation Distrubution Algorithm; Chow-Liu Tree; MIMIC.

Introduction

Being able to predict an outcome of a certain event will remain to be the most powerful tool humanity possesses. To be able to do this, we attempt to describe these events with mathematical models. Although these models cannot accurately predict a future events with 100% certainty, being able to deduce that an outcome is *likely* to occur, allows judging the situations an easier task.

Once an event is has been sampled over many iterations, we can observe patterns in the data they generate. We use known probability distributions that may follow these patterns. To be able to justify why a certain distribution *fits* the observed data, we use maximum likelihood estimates, which tell us how likely it is to observe what was observed, if we assume a certain distribution is the source of the data. Maximizing this likelihood then leads to properly adjusted distributions.

In this document, we use Mutual-Information-Maximizing Input Clustering (MIMIC) and the Chow-Liu tree estimation distribution algorithms to minimize three objective functions, where the sampling at each generation is done from a normal random variable. Using the fact that the mutual information between two normal random variables is $-\frac{1}{2}\log(1-\rho^2)$, we may build the corresponding weights for the Chow-Liu tree or minimize this value for the MIMIC.

Models

MIMIC

This algorithm was first proposed in [1], it takes a combinatorial approach to the best model. That is, it produces a first-order dependency chain (a simple Bayesian network) to represent the joint probability. It finds the optimal ordering $X_{\pi(1)}, X_{\pi(2)}, ..., X_{\pi(n)}$ by minimizing the Kullback-Leibler divergence between the true distribution of the elite solutions and the estimated chain distribution. Mathematically, this is equivalent to finding the chain that maximizes the mutual information between adjacent variables.

Citation: Pedro Llerenas. Homework 4: EDA with MIMIC and Chow-Liu Tree using Normal Variables. **2026,** 15, October.

Algorithm 1: MIMIC

$X :=$ Random initial population of size N from uniform distribution
Evaluate fitness $f(x)$ for all $x \in X$
while not converged **do**
  $\theta_t :=$ Fitness threshold for top θ-percentile
  $E := \{x \in X \mid f(x) \geq \theta_t\}$ (Elite subset)
  Compute marginal $H(X_i)$ and conditional $H(X_i \mid X_j)$ entropies for all pairs in E
  Find permutation π to minimize total conditional entropy: $H\left(X_{\pi(1)}\right) + \sum_{i=1}^{n-1} H\left(X_{\pi(i+1)} \mid X_{\pi(i)}\right)$
  $X :=$ Sample $N-1$ new candidates from this dependency chain using a normal distribution
  Evaluate fitness $f(x)$ for all new $x \in X$
end
return Best candidate $x^* \in X$

Since we will assume the variables come from a normal distribution, we can make furthen simplifications on the minimization problem. That is, we know that the covariance matrix

$$\Sigma = \begin{pmatrix} \mathrm{Var}(X_1) & \mathrm{Cov}(X_1, X_2) \\ \mathrm{Cov}(X_2, X_1) & \mathrm{Var}(X_2) \end{pmatrix}$$

and that $\rho = \frac{\mathrm{Cov}(X_1, X_2)}{\sigma_1 \sigma_2}$, so $\mathrm{Cov}(X_1, X_2) = \rho \sigma_1 \sigma_2$, and we may write the covariance matrix as

$$\Sigma = \begin{pmatrix} \sigma_2^1 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \rho_2^2 \end{pmatrix}.$$

Since the conditional distribution of two normal variables is once again normal, the conditional variance becomes, using Schur's complement,

$$\begin{aligned} \mathrm{Var}(X_1 | X_2 = x_2) &= \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21} \\ &= \sigma_1^2 - (\rho\sigma_1\sigma_2)\left(\frac{1}{\sigma_2^2}\right)(\rho\sigma_1\sigma_2) \\ &= \sigma_1^2(1-\rho^2) \end{aligned}$$

Therefore, since the differential entropy of a normal variable is

$$H(X) = \frac{1}{2}\log(2\pi e\sigma^2),$$

we have

$$\begin{aligned} H\left(X_i \mid X_p\right) &= \frac{1}{2}\log\left(2\pi e\sigma_{i|p}^2\right) \\ &= \frac{1}{2}\log(2\pi e\sigma_i^2(1-\rho^2)). \end{aligned}$$

Since $\log$ is strictly increasing, it suffices to minimize $\sigma_i^2(1-\rho^2)$.

Moreover, we will be taking the top 50 percentile of the individuals for our elitism conditions.

Dependency Tree

This model is similar to MIMIC in their goal of maximizing mutual information (or minimize entropy). However, dependency tree does not restrict the number of children a node can have, producing a directed acyclic graph, where each variable has at most one parent.

The following algorithm is also a simplified version of dependency tree, where we do not consider elitism. It was first presented in [2].

Algorithm 2: Dependency Tree

$X :=$ Random initial population of size N
Compute Mutual Information $I(X_i; X_j)$ for all pairs of variables
Let $G = (V, E)$ be a complete graph where weight $W(i, j) = I(X_i; X_j)$
$T :=$ Maximum Spanning Tree of G (via Kruskal)
Select an arbitrary node $R \in V$ as the root
Direct all edges in T outward from R to form a dependency tree
Estimate marginal $P(R)$ and conditionals $P\left(X_i \mid X_{\text{parent}(i)}\right)$
return pointer to R (root of tree)

In this case, we know that the mutual information is given by $-\frac{1}{2}\log(1-\rho^2)$. However, we also note that $|\rho|$ has behaves the same way as the mutual information, in the sense that $|\rho|$ is increasing in the same region, and decreasing in the same region. Therefore, the weights of the Chow-Liu tree are set to $|\rho|$.

Experiment Setup

Experiment 1

We will use both MIMIC and Chow-Liu tree to minimize the function

$$f(x) = \sum_{i=1}^{d} x_i^2$$

for $d = 10$. We will run both models 10 times, with a different seed per execution.

Experiment 2

With both models, we minimize the function

$$f(x) = \sum_{i=1}^{d} \frac{x_i^2}{4000} - \prod_{i=1}^{d} \cos\left(\frac{x_i}{\sqrt{i}}\right) + 1$$

for $d = 10$. We will run the models 10 times, with a different seed per execution.

Experiment 3

With both models, we minimize the function

$$f(x) = -a \exp\left(-b\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2}\right) - \exp\left(\frac{1}{d}\sum_{i=1}^{d} \cos(cx_i)\right) + a + \exp(1)$$

for $d = 10$. We will run the models 10 times, with a different seed per execution.

Results

For 500 individuals, we obtained the following results:

Sphere Function

Algorithm	Best	Worst	Mean	Median	Std Dev	Avg. Generations
Chow Liu	1.2681e-10	2.33347e-10	1.67156e-10	1.46955e-10	4.18298e-11	57
MIMIC	1.03311e-08	2.20328e-08	1.49581e-08	1.39092e-08	3.84446e-09	58

Table 1: Statistics for the sphere function on both models, for 500 individuals.

Since we know that the variables have no dependencies, we do not present a graph of the resulting tree or chain, as we do not expect any pattern to arise.

Griewank Function

Since this function

Algorithm	Best	Worst	Mean	Median	Std Dev	Avg. Generations
Chow Liu	3.56959e-12	1.43332e-11	9.82802e-12	1.04839e-11	3.71945e-12	76
MIMIC	7.0552e-10	1.38498e-09	9.92723e-10	9.98306e-10	2.25661e-10	76

Table 2: Statistics for the Griewank function on both models, for 500 individuals.

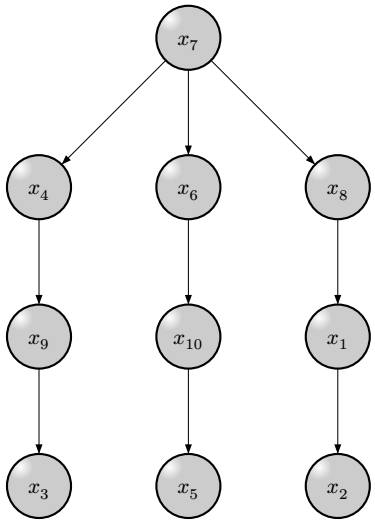


Figure 1: One of the trees generated by the Chow-Liu algorithm for the Griewank function.

Ackley Function

Algorithm	Best	Worst	Mean	Median	Std Dev	Avg.Generations
Chow Liu	1.23935e-05	1.83167e-05	1.55985e-05	1.57233e-05	1.80581e-06	47
MIMIC	0.000116792	0.000195041	0.000164091	0.000166689	2.51503e-05	48

Table 3: Statistics for the Ackley function on both models, for 500 individuals.

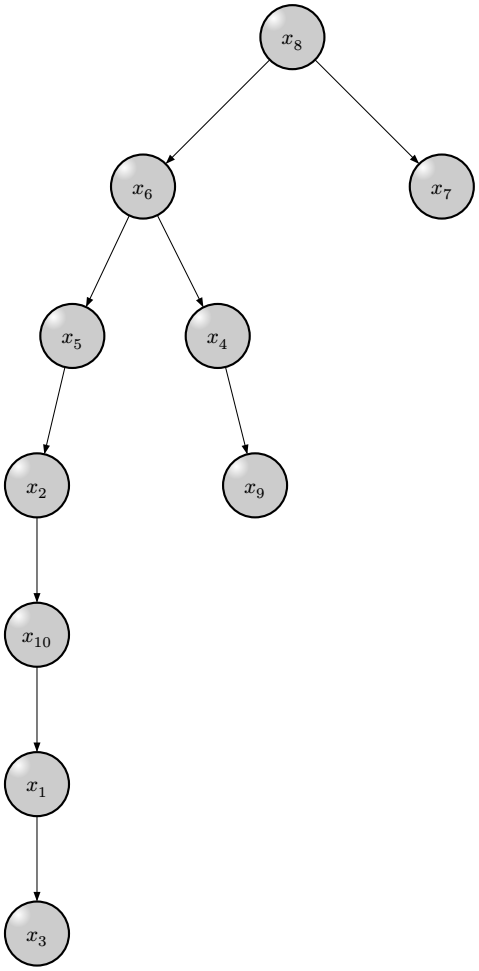


Figure 2: One of the trees generated by the Chow-Liu algorithm for the Ackley function.

Conclusions

We observe that the Chow-Liu algorithm reaches a better approximation of the true minimum of the objective function compared to the MIMIC model. Both models are able to get close to the global minimum.

Appendix

Compilation Instructions

Requirements:

- C++20
- CMake
- Make

First, `cd` into the directory. Then,

```
mkdir build && cd build
```

Then, we use CMake to generate the make files.

```
cmake ..
```

Finally, we use `make` to compile and obtain the necessary binary files.

```
make
```

Execution Instructions

All the results are obtained from a single execution, all printed to `cout`. Once all compilations are done, we execute (while in the `build` directory):

```
./src/main 500 10
```

The first argument is the population number, while the second is the number of executions per model and function.

This will print the tables presented in previous sections. If you also want to see all the trees and chains it generated on all 10 runs, you must un-comment the lines `MIMIC.cpp:85` and `ChowLiu.cpp:107` and re-compile.

Bibliography

[1] J. S. D. Bonet, C. L. Isbell, and P. A. Viola, "MIMIC: Finding Optima by Estimating Probability Densities," in *Neural Information Processing Systems*, 1996. [Online]. Available: https://api.semanticscholar.org/CorpusID:6126501

[2] C. Chow and C. Liu, "Approximating discrete probability distributions with dependence trees," *IEEE Transactions on Information Theory*, vol. 14, no. 3, pp. 462–467, 1968, doi: 10.1109/TIT.1968.1054142.